

Code Quality Document - Neighbourhood WatchDog

Project: COS 301 Capstone 2026

Team: Team Intrepid

Table of Contents

1. [Purpose](#)
 2. [Quality Tooling Overview](#)
 3. [Python Standards](#)
 4. [TypeScript & React Standards](#)
 5. [Git & Pull Request Standards](#)
 6. [Testing Standards](#)
 7. [CI/CD Quality Gates](#)
 8. [SonarCloud Standards](#)
 9. [Security Standards](#)
 10. [Current Compliance Status](#)
-

1. Purpose

This document defines the code quality standards for the Neighbourhood WatchDog project. All contributors are expected to adhere to these standards. Automated checks enforce the majority of the rules described below - contributions that fail any CI check will not be merged.

The goals of these standards are:

- **Consistency** - code reads the same way regardless of who wrote it
 - **Maintainability** - new contributors can understand and modify any part of the codebase
 - **Reliability** - automated testing and static analysis catch regressions before they reach production
 - **Security** - no credentials in source, no known vulnerable dependencies
-

2. Quality Tooling Overview

Tool	Scope	Enforced in CI
Ruff v0.4.4	Python linting & import checks	Yes
ESLint (Next.js config)	TypeScript/React linting	Yes
SonarCloud	Multi-language code quality & security hotspots	Yes
pip-audit	Python dependency vulnerability scan	Yes
Pytest	Backend unit & integration tests	Yes
Playwright	End-to-end browser tests	Yes
Branch protection	PR rules, merge restrictions	Yes

All tools run automatically on every pull request. A PR cannot be merged if any check fails.

3. Python Standards

Applies to: backend/ , ai/

3.1 Linter - Ruff

Ruff is the project's sole Python linter and import checker. The following rule categories are enforced:

Rule set	Examples enforced
E / W (pycodestyle)	Indentation, line length, whitespace
F (pyflakes)	Unused imports, undefined names
I (isort)	Import ordering
E401	No multiple imports on one line
F401	No unused imports

Run locally:

```
ruff check .
```

Auto-fix safe issues:

```
ruff check . --fix
```

3.2 Naming Conventions

Element	Convention	Example
Variables & functions	snake_case	detection_event , get_current_user
Classes	PascalCase	DetectionEvent , AuthMiddleware
Constants	UPPER_SNAKE_CASE	BACKEND_URL , CAMERA_ID
Private helpers	Leading _	_extract_detections() , _build_track_payload()
Modules / files	snake_case	detection_service.py , alert.py

3.3 Cognitive Complexity

SonarCloud enforces a maximum cognitive complexity of **15** per function. Functions that exceed this must be refactored by extracting focused helper functions. A helper function should do one thing, have a clear name, and be ≤ 20 lines.

Bad:

```
def _detection_loop():  
    # 80-line function with nested if/for/try blocks - complexity 29
```

Good:

```
def _detection_loop():  
    # Orchestrator: delegates to focused helpers - complexity < 10  
    ...  
  
def _collect_tracks(tracks, alerted_ids):  
    # Single responsibility - complexity < 5  
    ...
```

3.4 Exception Handling

- Use `logger.exception(...)` instead of `logger.error(...)` inside `except` blocks - it automatically attaches the traceback.

- Never use a bare `except:` - always catch a specific exception or at minimum `Exception`.
- Do not swallow exceptions silently unless the failure is explicitly non-critical (e.g. fire-and-forget network calls).

```
# Bad
except Exception as e:
    logger.error("Something failed: %s", e)

# Good
except Exception:
    logger.exception("Detection loop failed for camera %s", camera_id)
```

3.5 Type Hints

All public functions must have type-annotated parameters and return types. Private helper functions (`_` prefix) should be annotated where practical.

```
def _build_track_payload(track) -> dict:
    ...

async def get_clip_url(detection_event_id: str, db: DbSession, claims: dict) -> dict:
    ...
```

3.6 Docstrings

All public functions, classes, and modules must have a one-line or multi-line docstring. Private helpers should have at minimum a one-line description.

4. TypeScript & React Standards

Applies to: `frontend/`

4.1 Linter - ESLint

ESLint runs with `eslint-config-next/core-web-vitals` and `eslint-config-next/typescript`. This enforces React hooks rules, accessibility basics, and TypeScript best practices.

Run locally:

```
cd frontend && pnpm lint
```

4.2 Naming Conventions

Element	Convention	Example
Components	PascalCase	AlertCard , AlertFootagePlayer
Hooks	camelCase prefixed use	useClip , useCameraAnnotations
Utility functions	camelCase	formatDateTime , timeAgo
Types / interfaces	PascalCase	AlertCardProps , ClipStatus
Constants	UPPER_SNAKE_CASE	CLIP_PRE_EVENT_SECS
Files - components	PascalCase.tsx	AlertCard.tsx
Files - hooks	kebab-case.ts	use-clip.ts
Files - utilities	kebab-case.ts	alert.ts , auth.ts

4.3 Props - Read-only

All component prop interfaces must use `readonly` modifiers. This prevents accidental mutation and satisfies SonarCloud's TypeScript rules.

```
// Bad
export interface AlertCardProps {
  alert: Alert;
  onAcknowledge: (id: string) => Promise<void>;
}

// Good
export interface AlertCardProps {
  readonly alert: Alert;
  readonly onAcknowledge: (id: string) => Promise<void>;
}
```

4.4 Function Nesting Depth

SonarCloud enforces a maximum nesting depth of **4 levels**. Deeply nested callbacks inside WebRTC handlers, WebSocket listeners, or async effects must be extracted into named helper

functions.

```
// Bad - 5 levels deep inside pc.ontrack -> callback -> if -> canvas operations
pc.ontrack = (event) => {
  const video = videoRef.current;
  if (video) {
    video.onloadedmetadata = () => {
      if (video.videoWidth > 0) {
        setVideoWidth(video.videoWidth); // depth 5
      }
    };
  }
};

// Good - extracted helper keeps nesting <= 4
function applyVideoDimensions(video: HTMLVideoElement) {
  if (video.videoWidth > 0) {
    setVideoWidth(video.videoWidth);
    setVideoHeight(video.videoHeight);
  }
}

pc.ontrack = (event) => {
  const video = videoRef.current;
  if (!video) return;
  video.onloadedmetadata = () => applyVideoDimensions(video);
  video.srcObject = event.streams[0];
};
```

4.5 Browser Globals

Use `globalThis` instead of `window` for browser API access. Next.js renders components server-side where `window` is undefined; `globalThis` is safe in both environments.

```
// Bad
const protocol = window.location.protocol;

// Good
const protocol = (globalThis.location?.protocol ?? "http:") === "https:" ? "wss" : "ws";
```

4.6 Unused Imports

All unused imports must be removed. ESLint and Ruff both enforce this. Do not leave `import { Foo }` in a file if `Foo` is never referenced.

5. Git & Pull Request Standards

5.1 Branching Strategy

Branch	Purpose
main	Production-only. Direct commits are blocked.
dev	Integration branch. All features merge here first.
feature/<name>	New features, branched from dev .
fix/<name>	Bug fixes, branched from dev .

Rule enforced by CI: Pull requests targeting `main` are only accepted from the `dev` branch. Feature/fix PRs must target `dev` .

5.2 Commit Messages

Use the **Conventional Commits** format:

```
<type>(<scope>): <short description>

[optional body]
```

Type	When to use
feat	New feature
fix	Bug fix
docs	Documentation only
refactor	Code change with no behaviour change
test	Adding or updating tests
chore	Tooling, dependencies, config
ci	CI/CD workflow changes

Examples:

```
feat(ai): add rolling frame buffer for clip capture
fix(backend): correct userrole enum values in seed.sql
docs: update README badge consistency
refactor(frontend): extract applyVideoDimensions helper to reduce nesting
```

5.3 Pull Request Requirements

- At least **one peer review** is required before merging into `dev`
 - All CI checks must pass (lint, tests, SonarCloud quality gate)
 - PR description must reference the relevant GitHub issue
 - No direct commits to `main` or `dev`
-

6. Testing Standards

6.1 Backend - Pytest

Unit and integration tests live in `backend/tests/`. Each controller and service should have a corresponding test file.

```
backend/tests/
  test_alert.py
  test_clips.py
  test_detection.py
  ...
```

Run locally:

```
cd backend && pytest
```

Minimum coverage target: 80% line coverage on `backend/app/`.

6.2 End-to-End - Playwright

E2E tests live in `tests/e2e/` and cover critical user flows. The frontend must be running before tests execute. Configure `playwright.config.ts` with a `webServer` block so CI starts the server automatically.

Required E2E Coverage - Planned

Playwright infrastructure is present and invoked in CI. However, as of the verification date, the repository contains only the default Playwright example test. The following product flows are required before claiming end-to-end coverage:

Flow	Proposed test file	Status
Submit join request	joinNeighbourhood.spec.ts	Planned
Acknowledge alert	alert.spec.ts	Planned
Approve/deny join requests	request-page.spec.ts	Planned
View alert footage	alert-footage.spec.ts	Planned
Register, enable, and disable a camera	camera-lifecycle.spec.ts	Planned
Pair WatchDog Agent to property	agent-pairing.spec.ts	Planned

Run locally:

```
cd tests/e2e && npm test
```

6.3 Test Quality Rules

- Tests must be independent - no shared mutable state between tests
- Use descriptive test names: "resident cannot view footage from a private camera"
- Mock external services (S3, Cognito) in unit tests - never call real AWS in CI
- Each acceptance criterion in a user story should have at least one test case

7. CI/CD Quality Gates

The CI pipeline (`ci.yml`) runs three parallel jobs on every push and pull request:

`python-lint`

- Runs `ruff check` across the entire repository
- **Fails if:** any E/F/W/I rule violation is found
- Fix: `ruff check . --fix` for auto-fixable issues; manual fix otherwise

`frontend-lint`

- Runs `pnpm lint` (ESLint with Next.js config)
- **Fails if:** any ESLint rule violation is found

tests

- Starts a PostgreSQL service (PostGIS)
- Installs Python 3.11 and Node 20 dependencies
- Runs Pytest for backend
- Runs Playwright for E2E
- Runs pip-audit for dependency vulnerabilities

SonarCloud Analysis

- Triggered on every PR
- **Fails the quality gate if:**
 - Any new security hotspot
 - Cognitive complexity > 15 on any function
 - Nesting depth > 4 on any function/block
 - Any new blocker or critical issue

build

Builds the Next.js frontend with `pnpm build`. This catches production-build failures that linting alone cannot detect.

type-check

Runs Mypy against the backend with `--ignore-missing-imports`. Type-hint coverage and strictness remain improvement areas.

security

Runs Bandit against `backend/` and executes `pip-audit`. Dependency auditing must be revised to target the project requirement files explicitly.

A PR will not be merged if any of these gates are in a failed state.

8. SonarCloud Standards

SonarCloud analyses all languages in the repository. The project ID is `cos301-SE-2026_Neighbourhood-WatchDog`.

Quality Gate Conditions

Metric	Threshold
New security hotspots	0 (all must be reviewed)
Cognitive complexity	≤ 15 per function
Function nesting depth	≤ 4 levels
Duplicated lines	$< 3\%$ on new code
Maintainability rating	A

Addressing SonarCloud Findings

Blocker / Critical - must be fixed before merge.

Major - must be fixed or marked as accepted with a justification comment.

Minor / Info - fix if practical; may defer.

Common Fixes

Finding	Fix
Cognitive complexity > 15	Extract helper functions
Nesting depth > 4	Extract named helpers, use early returns
Non-readonly props	Add <code>readonly</code> to interface fields
<code>window</code> reference	Replace with <code>globalThis</code>
<code>logger.error</code> in except	Replace with <code>logger.exception</code>
Hardcoded Docker base tag	Pin to a specific version tag (e.g. <code>alpine:3.21</code>)
Commented-out code	Remove it - version control is the history

9. Security Standards

9.1 Secrets & Credentials

- **Never commit** API keys, access tokens, database passwords, or AWS credentials to the repository
- All secrets live in `.env` at the project root
- `.env` is listed in `.gitignore` - confirm this before every commit
- Use `.env.example` (without real values) to document required variables

9.2 Dependency Security - pip-audit

`pip-audit` scans Python dependencies for known CVEs on every CI run. If a vulnerability is found:

1. Update the affected package to the patched version in `requirements.txt`
2. Re-run `pip-audit` locally to confirm the fix
3. Push the update and re-trigger CI

Run locally:

```
pip-audit
```

9.3 Docker Images

- Pin all Docker base images to a specific version tag - never use `:latest`
- Example: `FROM alpine:3.21` NOT `FROM alpine:latest`
- This ensures reproducible builds and prevents silent base-image changes

9.4 RBAC - Footage & Stream Access

All camera footage and live-stream access is subject to role-based access control:

Role	Live streams	Footage clips
SYSTEM_ADMIN	All cameras	All cameras
NEIGHBOURHOOD_ADMIN	All cameras in neighbourhood	All cameras in neighbourhood
PROPERTY_ADMIN	All cameras in neighbourhood	All cameras in neighbourhood
RESIDENT	Public cameras in their neighbourhood	Public cameras in their neighbourhood

The RBAC check must be applied server-side - the frontend must never gate access based solely on client-side role checks.

10. Current Compliance Status

The table below distinguishes controls verified in the repository from standards that remain targets or planned improvements.

Standard / Control	Status	Evidence / Limitation
Ruff Python linting	Implemented	<code>python-lint</code> runs <code>ruff check</code> in CI. The exact enabled rule families must be documented from an explicit Ruff configuration before claiming <code>w</code> or <code>I</code> enforcement.
ESLint TypeScript linting	Implemented	<code>frontend-lint</code> runs <code>cd frontend && pnpm lint</code> using Next.js Core Web Vitals and TypeScript configurations.
Frontend production build	Implemented	<code>build</code> runs <code>pnpm build</code> in the frontend directory.
Backend type checking	Implemented	<code>type-check</code> runs <code>mypy . --ignore-missing-imports</code> from <code>backend/</code> .
Backend Pytest suite	Implemented	<code>tests</code> provisions PostGIS, runs migrations and seed data, then runs Pytest from <code>tests/backend/</code> with coverage collection.
Frontend Jest suite	Implemented	<code>tests</code> runs <code>npx jest --coverage</code> from <code>frontend/</code> .
Product E2E coverage	In Progress	Playwright is installed and executed in CI, but the current suite contains only the default <code>example.spec.ts</code> ; product acceptance flows still need implementation.
Bandit security scan	Implemented	<code>security</code> runs <code>bandit -r backend/ -ll</code> .
Python dependency vulnerability audit	Partial	<code>pip-audit</code> runs in CI, but the workflow should explicitly audit <code>backend/requirements.txt</code> and <code>ai/requirements.txt</code> .
Codecov upload	Implemented / verify repository	CI uploads coverage reports to Codecov; whether it blocks merges must be verified in

Standard / Control	Status	Evidence / Limitation
	settings	Codecov/GitHub settings.
SonarCloud quality gate	Planned	No SonarCloud analysis step or repository configuration was found in the current CI workflow.
Branch-source validation	Implemented	<code>branch-protection.yml</code> rejects PRs to <code>main</code> unless they originate from <code>dev</code> .
GitHub branch protection	Unverified	GitHub repository rulesets are configured outside the codebase and must be checked in repository settings before claiming enforcement.
Conventional Commit validation	Manual	No commitlint or equivalent enforcement is configured.
80% backend coverage threshold	Target	Coverage is collected, but no CI fail-under threshold is configured.
Server-side stream and WebSocket authorisation	In Progress	Browser playback and real-time WebSocket authorisation require further hardening before production claims can be made.
Private EC2 key handling	Non-compliant - urgent	A <code>watchdog-ec2-key.pem</code> file is present in the repository workspace. Confirm whether it is tracked, remove it from Git history if necessary, and rotate the corresponding EC2 key pair.

Recommended Future Improvements

1. **Add commitlint to CI** - enforce Conventional Commits format automatically
2. **Add coverage threshold** - fail CI if line coverage drops below 80%
3. **Add dependabot** - automated dependency update PRs for security patches
4. **Add pre-commit hooks** - run Ruff and ESLint locally before a commit reaches CI